Vision Lego

Developer Blueprint — Your personal reference. Not for Claude Code.

The big picture

You're building a fullstack web app called **Vision Lego** — an online eyewear store with two sides:

- A **public store** where customers browse and buy glasses
- A **private dashboard** at `/negocio` where the store owner manages everything

The whole thing lives in one SvelteKit project on your laptop. Claude Code writes the code, you review it and guide it. When you're happy, you push to GitHub and it goes live automatically.

Your tools and what they do

Tool	What it is	Why you're using it
SvelteKit	The web framework	Handles both frontend and backend in one project
TypeScript	Typed JavaScript	Catches errors before they happen
pnpm	Package manager	Faster than npm, installs dependencies
Drizzle	Database ORM	Talk to PostgreSQL using TypeScript instead of raw SQL
PostgreSQL	Database	Stores all your data (products, orders, users, etc.)
Better Auth	Auth library	Handles login, sessions, and user roles
Tailwind CSS	CSS utility framework	Styling without writing custom CSS files
MercadoPago	Payment provider	Card payments and PSE in Colombia
Vitest	Unit testing	Tests individual functions and components
Playwright	End-to-end testing	Tests the app like a real user in a browser
PM2	Process manager	Keeps the Node.js server running 24/7 on the droplet
Nginx	Reverse proxy	Receives internet traffic and forwards it to your app
GitHub Actions	CI/CD automation	Automatically deploys when you push to main

How the code is organized

```
vision_lego/

■■■ src/

■   ■■■ lib/

■   ■   ■■■ server/db/

■   ■   ■   ■■■ schema.ts     <- All database tables defined here
```

```
■   ■   ■   ■■■ index.ts      <- DB connection
■   ■   ■■■ components/       <- Reusable UI pieces
■   ■   ■■■ utils/            <- Helper functions (e.g. format prices)
■   ■■■ routes/               <- Every folder here = a URL
■       ■■■ (store)/          <- Public store pages
■       ■■■ (auth)/           <- Login / register pages
■       ■■■ negocio/          <- Owner dashboard (password protected)
■■■ static/                   <- Public files (images, fonts)
■■■ .env                      <- Your secret keys (NEVER commit this)
■■■ drizzle.config.ts         <- Drizzle migration config
■■■ ecosystem.config.cjs      <- PM2 config (Claude Code generates this)
■■■ nginx.conf                <- Nginx config (Claude Code generates this)
■■■ .github/workflows/
    ■■■ deploy.yml            <- Auto-deploy script (Claude Code generates this)
```

Key rule: anything in `src/lib/server/` never reaches the browser. That's where DB queries, auth, and secret keys live.

The database

PostgreSQL stores all the app's data. You talk to it through **Drizzle**, which means you write TypeScript instead of SQL. The schema lives at:

```
src/lib/server/db/schema.ts
```

Your tables at a glance

Table	What it stores
users	Everyone with an account (customers + owner)
sessions	Active login sessions
products	Sunglasses and optical frames
productVariants	Each color / size / material combo of a product
productImages	Photos per product or variant
brands	Eyewear brands
cartItems	Items a customer has in their cart
wishlistItems	Items a customer saved for later
orders	Every purchase made

orderItems	The individual products inside each order
expenses	Business expenses the owner logs
receivables	Outstanding debts / accounts receivable
shippingMethods	Delivery options and their costs

Migrations

When you change `schema.ts`, Claude Code syncs it to the real database:

```
pnpm drizzle-kit generate   # creates a migration SQL file

pnpm drizzle-kit migrate    # applies it to the database
```

You don't need to write SQL manually — ever.

Authentication and roles

Everyone logs in at `/iniciar-sesion`. Better Auth checks who they are and redirects accordingly:

Role	Where they go	What they can do
customer	Store pages	Browse, buy, manage their account
owner	/negocio	Full business dashboard
staff	/negocio	Same as owner (ready for future use)

The owner account **doesn't have a public register form** — you create it directly in the database (Claude Code will tell you how when the time comes).

Environment variables

Secret values your app needs to run. They live in a `.env` file at the project root. **Never commit this file to GitHub.**

```
# PostgreSQL connection string

DATABASE_URL=postgresql://vision_lego_user:your_password@localhost:5432/vision_lego



# Long random string — Better Auth uses this to sign sessions

BETTER_AUTH_SECRET=generate_a_long_random_string_here



# Where your app runs locally

BETTER_AUTH_URL=http://localhost:5173
```

For production, these same values are stored as **GitHub Actions secrets** so they're injected securely during deployment — never hardcoded anywhere.

The server (DigitalOcean Droplet)

What	Details
OS	Ubuntu 24 LTS
Node.js	v24.14.1
Nginx	Latest — receives internet traffic on ports 80 / 443
PostgreSQL	Latest — stores all your data
PM2	Keeps your Node.js app running 24/7
Your app	Runs on port 3000 (internal, not exposed to the internet)

How traffic flows:

```
Browser → Nginx (port 443/80) → your app (port 3000) → PostgreSQL
```

Nginx is the public face. It handles HTTPS and forwards requests to your app running quietly on port 3000.

One-time server setup (do this once, before first deploy)

SSH into your droplet, then run these commands in order.

1. Set up PostgreSQL

```
sudo -u postgres psql
```

Then inside psql:

```
CREATE USER vision_lego_user WITH PASSWORD 'choose_a_strong_password';

CREATE DATABASE vision_lego OWNER vision_lego_user;

GRANT ALL PRIVILEGES ON DATABASE vision_lego TO vision_lego_user;

\q
```

2. Install PM2

```
npm install -g pm2
```

After your first successful deploy:

```
pm2 startup    # copy and run the command it prints

pm2 save       # saves the process list so it survives reboots
```

3. Configure Nginx

Claude Code generates an `nginx.conf` file. Copy it to the server:

```
sudo cp nginx.conf /etc/nginx/sites-available/vision_lego

sudo ln -s /etc/nginx/sites-available/vision_lego /etc/nginx/sites-enabled/

sudo nginx -t                  # should print "ok"

sudo systemctl reload nginx    # apply it
```

One-time GitHub setup (do this once)

Go to your repo → **Settings** → **Secrets and variables** → **Actions** → **New repository secret** and add:

Secret name	Value
`DROPLET_HOST`	Your droplet's IP address
`DROPLET_USER`	mikey (non-root user with SSH access)
`DROPLET_SSH_KEY`	Your private SSH key (entire file content)
`DATABASE_URL`	postgresql://vision_lego_user:password@localhost:5432/vision_lego
`BETTER_AUTH_SECRET`	Same long random string as in your local .env
`BETTER_AUTH_URL`	Your domain, e.g. https://visionlego.co

The development workflow (day to day)

```
1. Go to the project folder

   cd vision_lego



2. Start the local dev server

   pnpm dev

   -> App runs at http://localhost:5173



3. Ask Claude Code to build a feature

   -> It writes the code, you review it



4. Test it locally in your browser



5. When happy, commit and push

   git add .

   git commit -m "add product catalog page"

   git push origin main

   -> GitHub Actions deploys automatically (~1-2 min)

   -> Your live site updates
```

That's the full cycle. You never manually copy files to the server or restart anything — it all happens automatically after `git push`.

What happens after git push

```
You push to main on GitHub

        |

GitHub Actions starts (runs on GitHub's servers, not yours)

        |

Runs pnpm build on your code

        |

SSHes into your droplet

        |

Copies build/ folder to the droplet

        |

Runs pnpm install --prod (production deps only, no build)

        |

Runs pm2 restart vision_lego

        |

Your app is live with the new version  [OK]
```

When the app is done — adding MercadoPago

MercadoPago is the last step intentionally. Everything else works without it (orders are created with status `pending`). When you're ready:

1. The store owner creates a MercadoPago account and shares their API credentials with you.
2. You add the MercadoPago MCP to Claude Code:

```
claude mcp add -t http -s project mercadopago https://mcp.mercadopago.com/mcp \

  --header "Authorization: Bearer OWNER_ACCESS_TOKEN"
```

3. Claude Code integrates **Checkout Bricks (Payment Brick)** into the checkout page.
4. Payments go live — customers can pay with card, Efecty, or PSE.

Quick reference — useful commands

```
# Local development

pnpm dev                      # start dev server at localhost:5173

pnpm build                    # build for production
```

```
pnpm preview                    # preview the production build locally


# Database
pnpm drizzle-kit generate       # generate migration after schema changes
pnpm drizzle-kit migrate        # apply migrations to the database
pnpm drizzle-kit studio         # open visual DB browser at localhost:4983


# Testing
pnpm test                       # run Vitest unit tests
pnpm test:e2e                   # run Playwright end-to-end tests


# Deploy
git push origin main            # triggers automatic deployment


# SSH into droplet
ssh -i ~/.ssh/Digital_Ocean_Key mikey@YOUR_DROPLET_IP


# Check app status on droplet
pm2 status
pm2 logs vision_lego            # view live logs
pm2 restart vision_lego         # manually restart if needed
```